6. Error detection and correction using Hamming Code Concept.

Aim:

To write a program to implement error detection and correction using Hamming code Concept. Make a test run to input data stream and verify error correction feature.

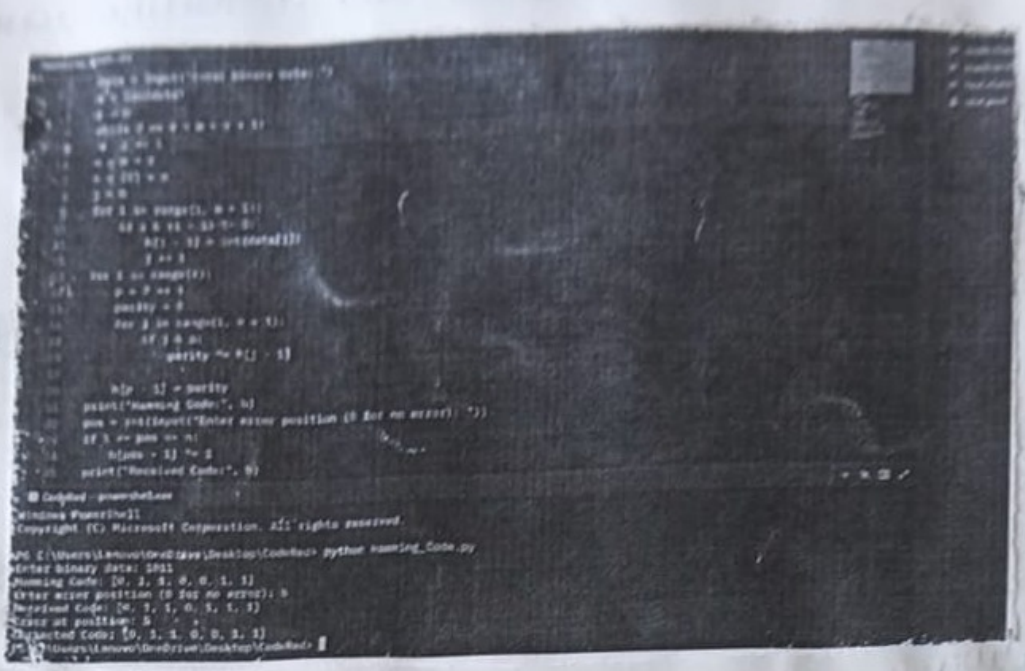
Error Correction at Data Link Layer:-

Hamming Code is a set of error-correction codes that can be used to detect and correct the error that can occur when the data is transmitted from the sender to the receiver. It is a technique developed by R. W Hamming for error correction.

Create Sender program with below feature:

1) Input to Sender file should be a text of any length. program should convert the text to binary.

2) Apply Hamming Code Concept on the binary data and add redundant bits on it.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Lenovo\OneDrive\Desktop\CodeRed> python Hamming_Code.py
Enter Binary Data: 1011
Hamming Code: [0, 1, 1, 0, 0, 1, 1]
Enter error position (0 for no error): 5
Received Code: [0, 1, 1, 0, 1, 1, 1]
Error at position: 5
Corrected Code: [0, 1, 1, 0, 0, 1, 1]
PS C:\Users\Lenovo\OneDrive\Desktop\CodeRed>
```

» Save this output in a file called channel.

create a receiver program with below features

1) Receiver program should read read the input from channel file.

2) Apply Hamming Code on the binary data to check for error.

» If there is an error, display the position of the error.

4) Else remove the reductan bits and convert the binary data to ascii and display the output.

Code:

```
def hamming-encode (data):
    encoded = " "
for i in range (0, len (data), 4),
        d = data[i:i+4]
    if len(d) < 4:
        d = d.ljust (4, '0')
d1, d2, d3, d4 = map (int, d)
```

$P_1 = d_1 \wedge d_2 \wedge d_4$

$P_2 = d_1 \wedge d_3 \wedge d_4$

$P_4 = d_2 \wedge d_3 \wedge d_4$

```
code = str(p1) + str(p2) + str(d1) + str(p4) + str(d2)
                                    + str(d3) + str(d4)

encode += code.

return encoded

text = input ("Enter text:")

binary = " ".join( format ( ord(c), '08b' ) for c in
                    text )

print("Binary data": .binary )

encoded_data hamming - encode (binary )

with open ("Channel.txt", "w") as file:

    file . write (encoded_data )

print ("Hamming encoded data :", encoded 'data)

print ("Data Saved to Channel.txt")
```

Receiver program:

→ Reads data from channel.txt

→ Checks every 7 bit hamming block.

→ Finds the error position using syndrome

→ Corrects the error or removes parity bits

→ Converts binary back to ASCII text.

Code:

```
def hamming_decode(data):
    Corrected = ""
    error_found = False
    for i in range(0, len(data), 7):
        block = list(map(int, data[i, i+7]))
        if len(block) < 7:
            Continue
        p1, p2, p3, p4, d1, d2, d3, d4, block
        s1 = p1 ^ d1 ^ d2 ^ d4
        s2 = p2 ^ d1 ^ d3 ^ d4
        s4 = p4 ^ d2 ^ d3 ^ d4
    error_position = s1 + (s2*2) + (s4*4)
    if error_position != 0:
        error_found = True
        actual_position = i + error_position
        print("Error detected at bit position", actual_pos)
```

```
        block [error position - 1] ^= 1
        print ("Error corrected at bit position :", actual
                - position)
    Corrected += str (block [2])
    corrected += str (block[4])
    Corrected += str (block [5]).
    Corrected += str [block [6])
    return Corrected, error - found
with open ("channel.txt", "r") as file
    Channel_data = file .read().strip()
print ("Data received from channel:")
print (channel data).
binary_data, error_found = hamming_decode (
                                channel_data )
text = ""
for i in range (0, len ( binary_data), 8).
    byte = binary_data [i:i+8]
    if len (byted ) == 8
        text += chr (int (byte, 2))
```

```
print("\n Corrected binary data :")
print (binary_data)
if error_found
    print ("\n Error was detected and corrected.")
else.
    print ("\n NO error detected .")
print ( "Received text :", text).
```

Input:

HELLO

~~Input~~ output:

Enter the text: HELLO

Error detected at position: 1

Error Corrected

Original Message: HELLO.

Result:

Therefore, the Error detection and Error Correction using Hamming code was executed Successfully.